

LINKSFOUNDATION.COM



**Politecnico
di Torino**

Applied Data Science Project

Vision-Language Models

FEDERICO D'ASARO

APPLIED DATA SCIENCE PROJECT, A.Y. 2025/2026

Table of Contents

1. Introduction to Vision-Language Models (VLMs)
2. From Unimodal Pre-training to Multimodal Pre-training
3. VLM Components and Training Objectives
4. Zero-Shot Tasks with VLMs
5. Embedding Space Structure of VLMs



WHY VISION-LANGUAGE MODELS?



What is Multimodality?

Definition: Multimodality refers to the integration and processing of information from multiple data types—or modalities—such as text, images, audio.

In AI, multimodal models learn to understand and relate information across these different modalities (e.g., matching an image with its caption or describing a video in text).

We focus on: Vision-Language Models (Image, Text modalities).



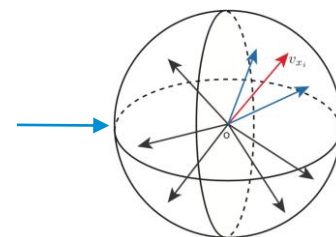
Why Vision-Language Models?

Motivation: Traditional computer vision systems can recognize objects and patterns in images, but they are limited: they do not understand natural language and cannot easily connect text with images.

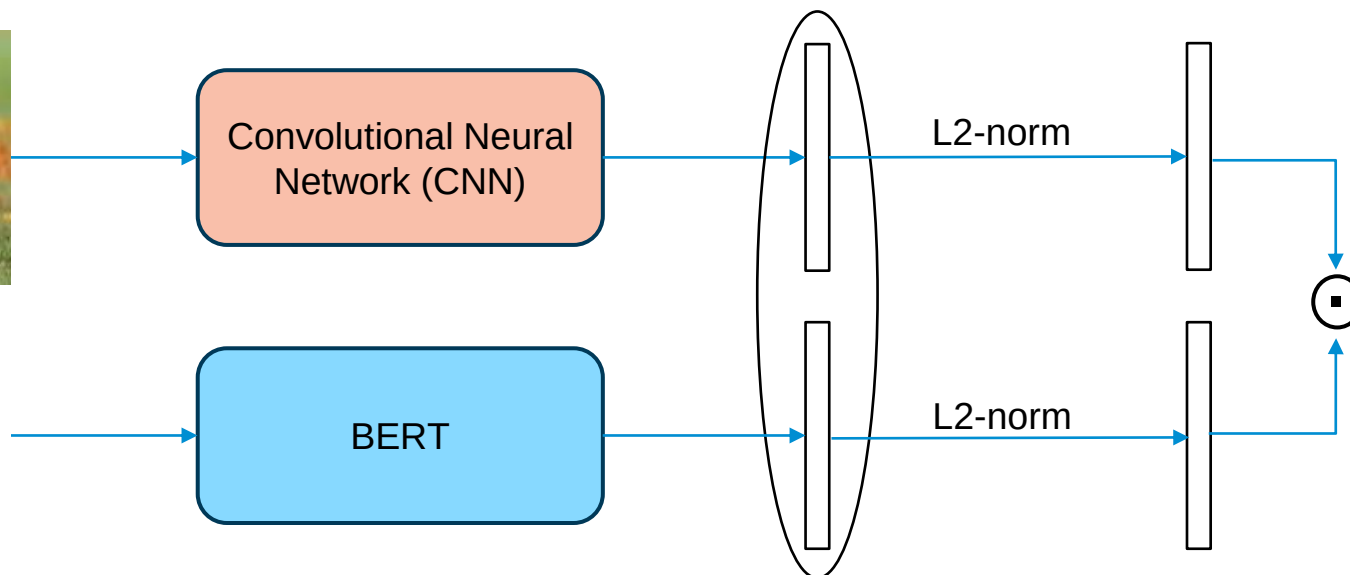
Consider two separate unimodal encoders—CNN for vision and BERT for language—each encoding semantically related concepts in different modalities.

Suppose both inputs are encoded as vectors of the same dimensionality, d .

To compare them, we often use cosine similarity—the dot product of L2-normalized embeddings.



a picture of a dog sitting on the grass



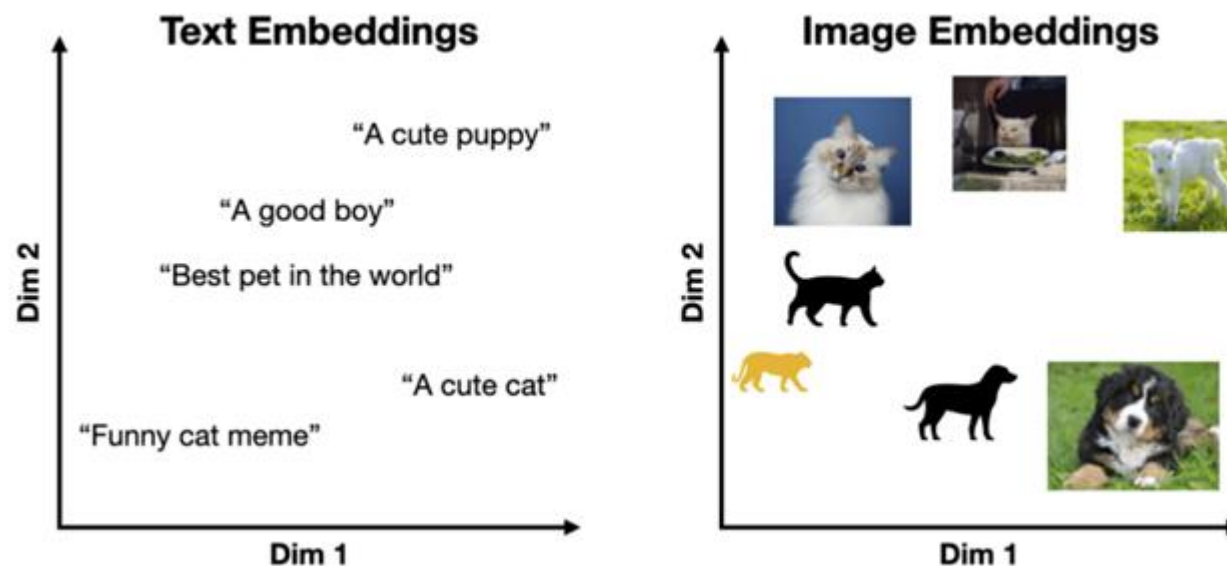
Here the cosine similarity will be near 0, indicating a very low score (range: -1 to 1).

Why Low Similarity?

The score is low because, when you use **separate unimodal encoders** (e.g., CNN for images and BERT for text), the embeddings are learned in **different representational spaces**.

This means:

- The vector for the *image* and the vector for the *text* don't share a common geometry.
- Even if they describe the same concept (e.g., "dog on grass"), their embeddings are essentially **not aligned**.
- As a result, the cosine similarity between them tends to be close to **0** (no correlation), rather than high.



Why Vision-Language Models?

Main Purpose of VLMs: VLMs such as CLIP are created to **unify vision and language**, enabling models to:

- Understand textual descriptions and associate them with visual content.
- Reason across modalities (text ↔ image).
- Be applied to flexible tasks without requiring task-specific training.



FROM UNIMODAL PRE-TRAINING TO MULTIMODAL PRE-TRAINING



Supervised Pre-training, Fine-tuning and Prediction

1. **Pre-training** on large labeled vision datasets (e.g., [ImageNet](#)).

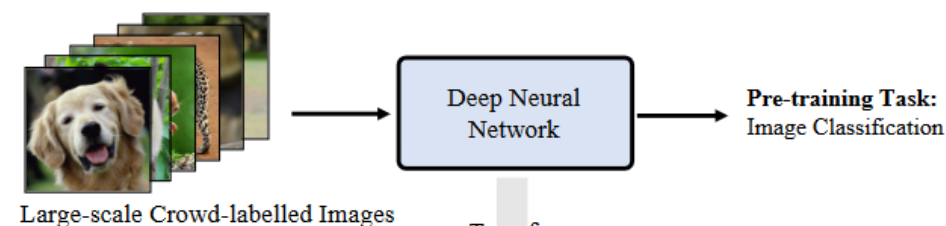
2. **Fine-tuning** on downstream visual tasks. Examples:



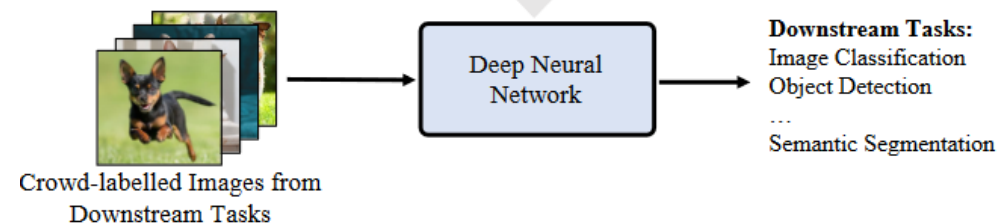
3. **Prediction** on unseen images.

(a). Supervised Pre-training, Fine-tuning and Prediction

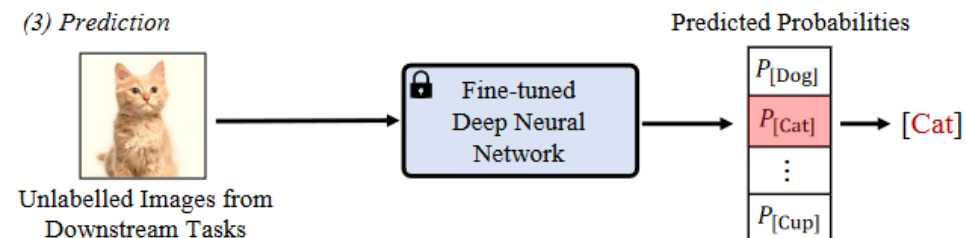
(1) Supervised Pre-training



(2) Task-specific Fine-tuning

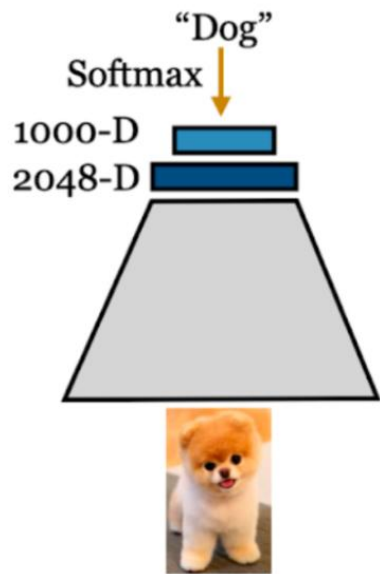


(3) Prediction



Supervised Pre-training, Fine-tuning and Prediction

Supervised pretraining is typically done on the ImageNet dataset—a large-scale benchmark containing over a million images across 1,000 object categories—using a cross-entropy loss.

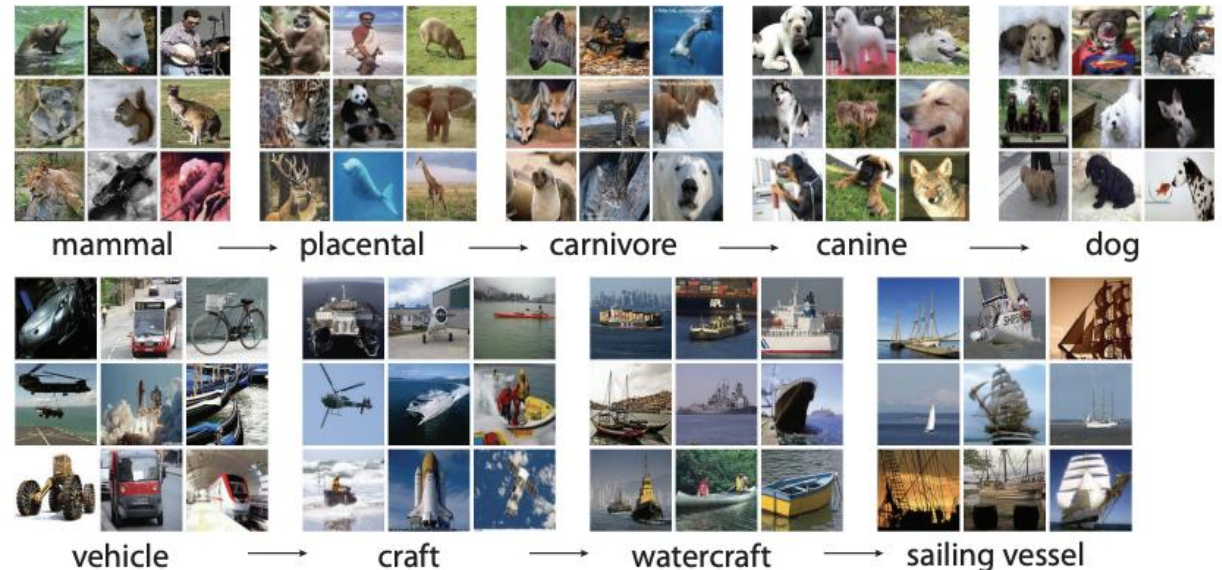


Cross-Entropy loss

$$f(s)_i = \frac{e^{s_i}}{\sum_j^C e^{s_j}}$$

$$CE = - \sum_i^C t_i \log(f(s)_i)$$

ImageNet



Unsupervised Pre-training, Fine-tuning and Prediction

1. **Pre-training** on large unlabeled vision datasets (e.g., [ImageNet](#) ignoring class annotations). The pretraining task is commonly called a Pretext Task.

2. **Fine-tuning** on downstream visual tasks.

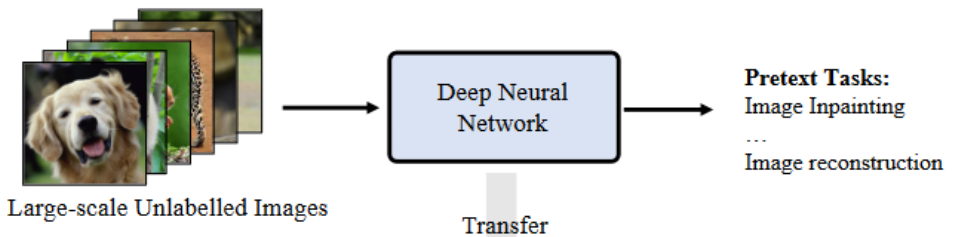


3. **Prediction** on unseen images.

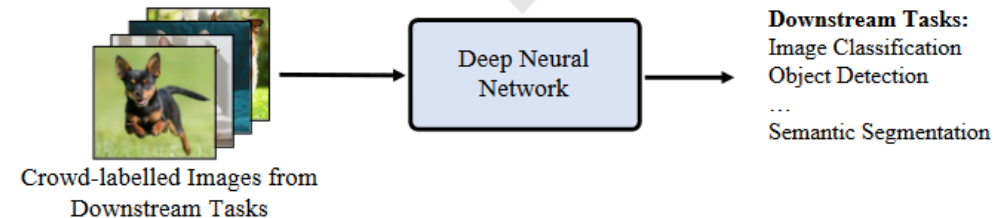
Since this paradigm does not require labelled data in pre-training, it can exploit more training data for learning useful and transferable features, leading to even better performance as compared with the supervised pre-training.

(b). Unsupervised Pre-training, Fine-tuning and Prediction

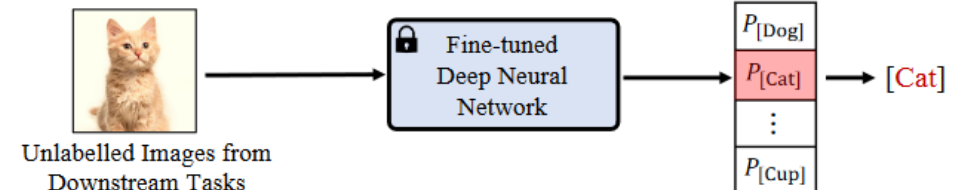
(1) Unsupervised Pre-training



(2) Task-specific Fine-tuning



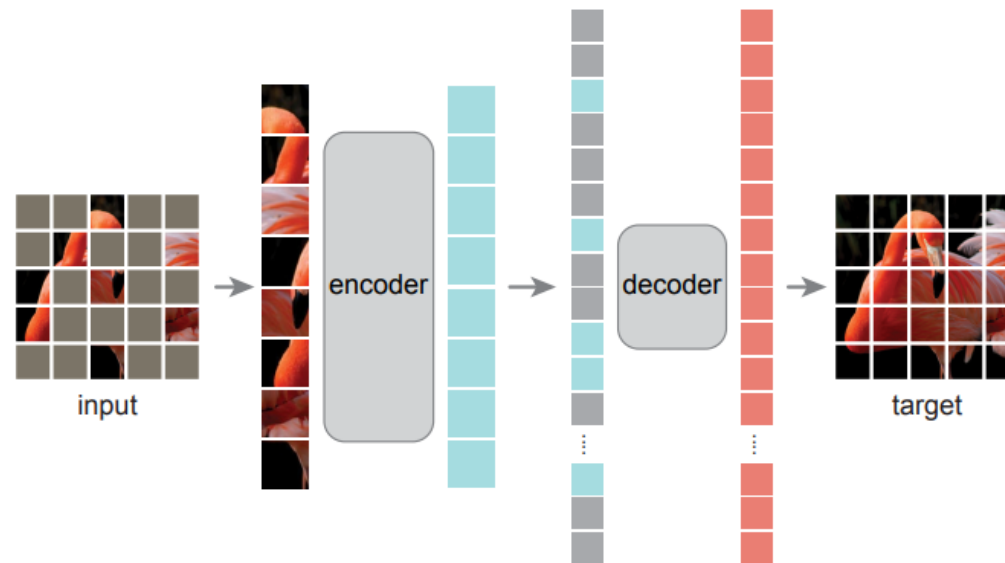
(3) Prediction



Examples of Pretext Tasks

MAE: Masked Autoencoders

- During pre-training, most image patches (e.g., 75%) are masked.
- The encoder processes only the visible patches.
- Mask tokens are added after encoding, and a lightweight decoder reconstructs the full image.
- After pre-training, the decoder is discarded; the encoder is used on complete images for recognition tasks.



Examples of Pretext Tasks

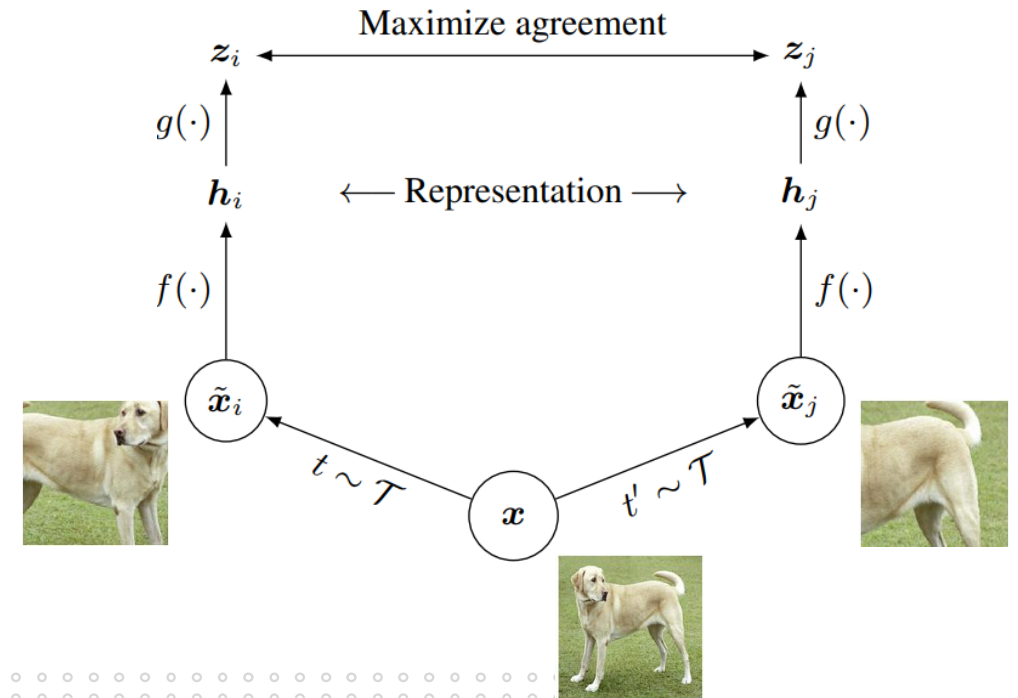
SimCLR: A contrastive learning approach

- Two augmented views of the same image are generated.
- An encoder and projection head are trained to maximize agreement using contrastive loss.
- After training, the projection head is discarded and the encoder's representations are used for downstream tasks.

Loss function:

- Each minibatch of N images produces $2N$ augmented views.
- For a positive pair, the other $2(N-1)$ views act as negatives.
- The loss maximizes cosine similarity of positives while minimizing it for negatives, scaled by a temperature parameter τ .

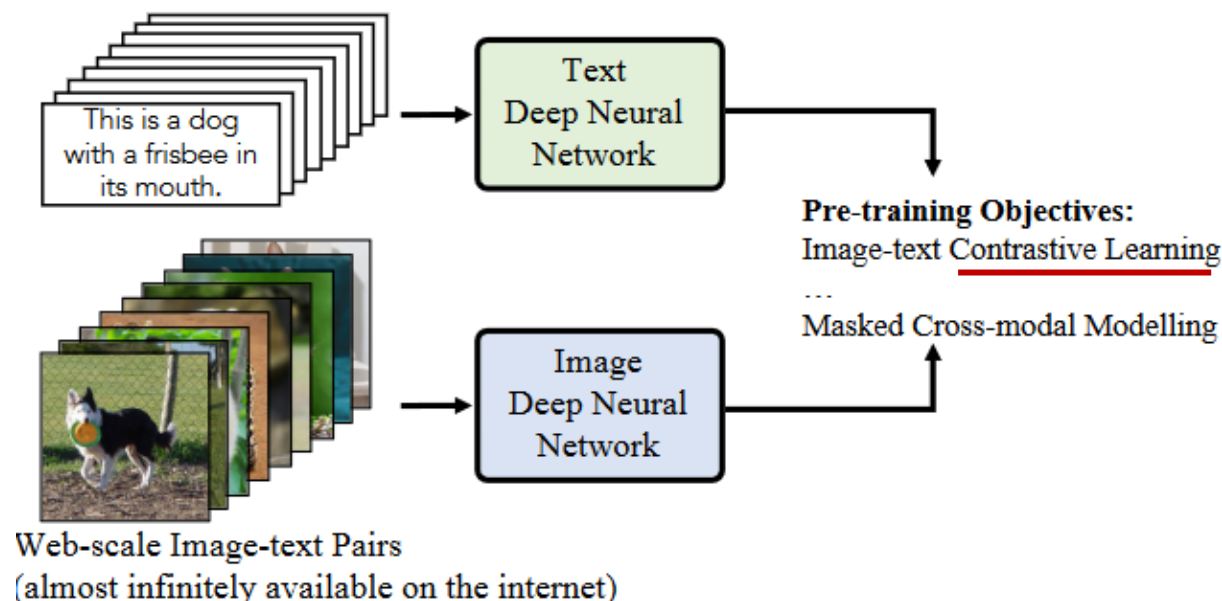
$$\ell_{i,j} = -\log \frac{\exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_j)/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{[k \neq i]} \exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_k)/\tau)}$$



Vision-Language Model Pre-training and Zero-shot Prediction

- 1. Pretraining:** Vision–Language Models (VLMs) are trained on large-scale image–text pairs from the web using vision-language objectives.

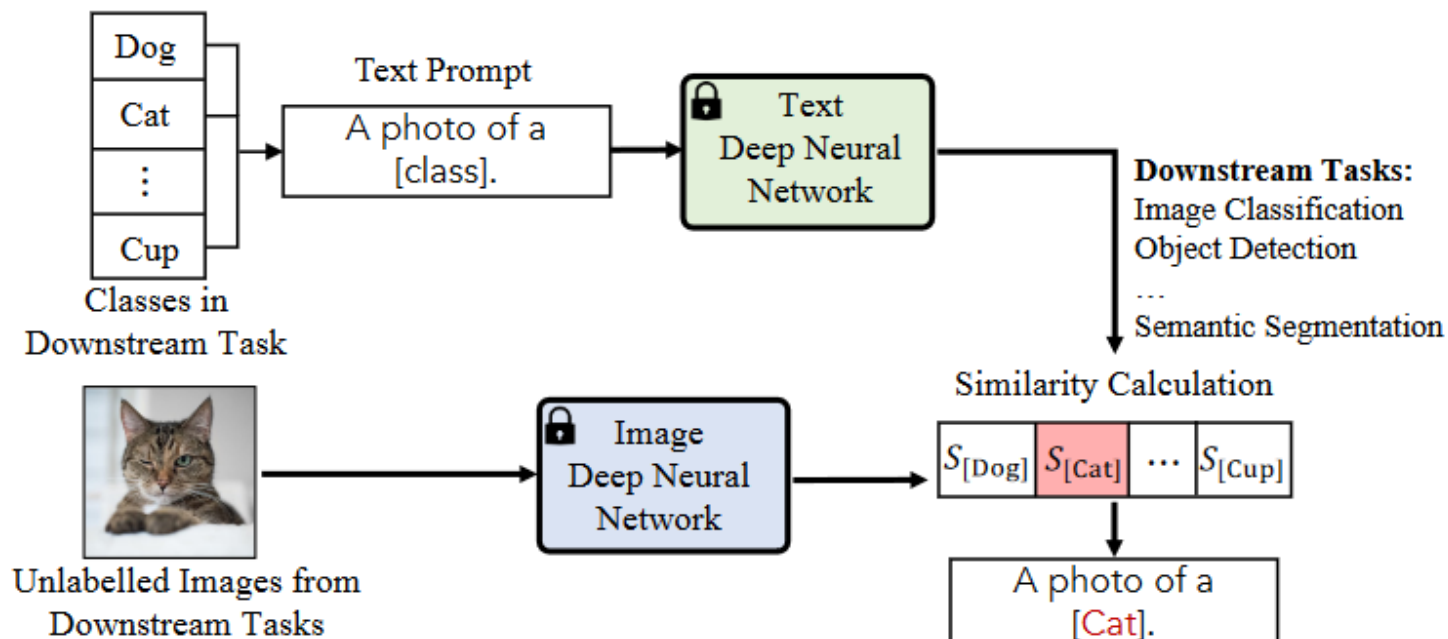
(1) Vision-Language Model Pre-training



Vision-Language Model Pre-training and Zero-shot Prediction

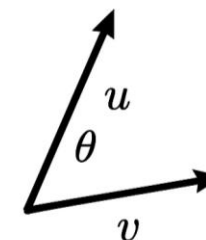
2. Prediction: pretrained VLMs learn strong vision–language alignment and can perform zero-shot predictions without additional fine-tuning.

(2) Zero-shot Prediction without Fine-tuning



Cosine Similarity

$$\text{sim}(u, v) = \frac{u \cdot v}{\|u\| \|v\|}$$



Cosine similarity is simply the dot product of L₂-normalized embeddings



VLM COMPONENTS AND TRAINING OBJECTIVES



VLM Components and Training Objectives

Components of VLMs:

- Image Encoder: Typically a CNN (ResNet) or Vision Transformer (ViT). Converts an image into a fixed-length vector (embedding).
- Text Encoder: Usually a Transformer-based language model. Converts text (captions, prompts, class labels) into embeddings.
- Contrastive Learning Objective: Aligns paired image–text embeddings. Pushes matched pairs closer, mismatched pairs further apart. Loss often uses cosine similarity + temperature scaling.

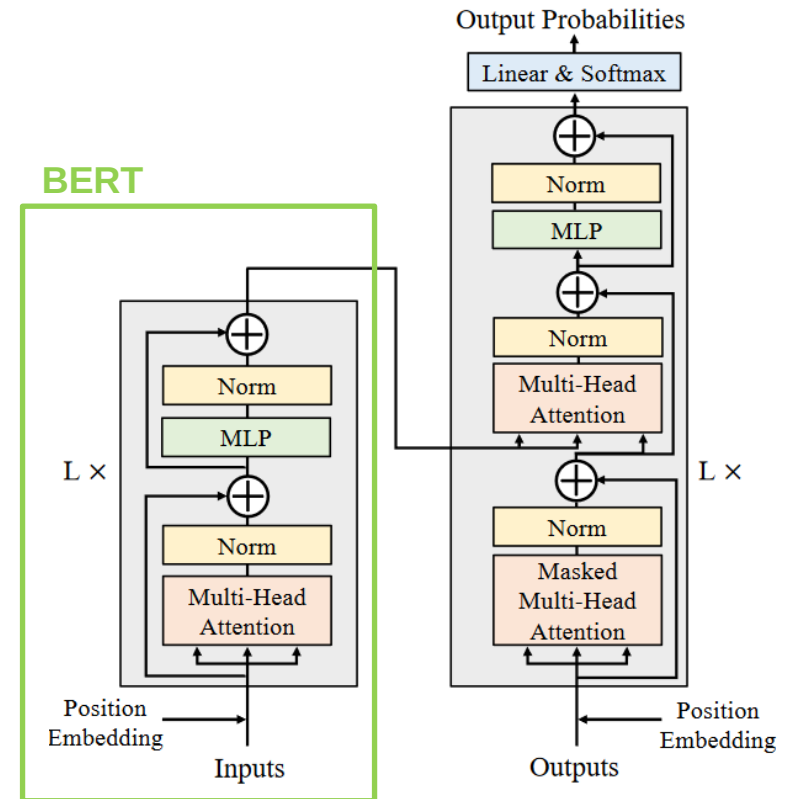
Key Characteristics of VLMs:

- Multimodal Alignment: Both image and text embeddings are projected into the same latent space. Enables direct comparison (similarity calculation).
- Zero-Shot Learning: Can classify or retrieve without task-specific training (just by comparing to text prompts).
- Scalable Training: Trained on hundreds of millions of image–text pairs (web data).

VLMs - Language components

Standard Transformer

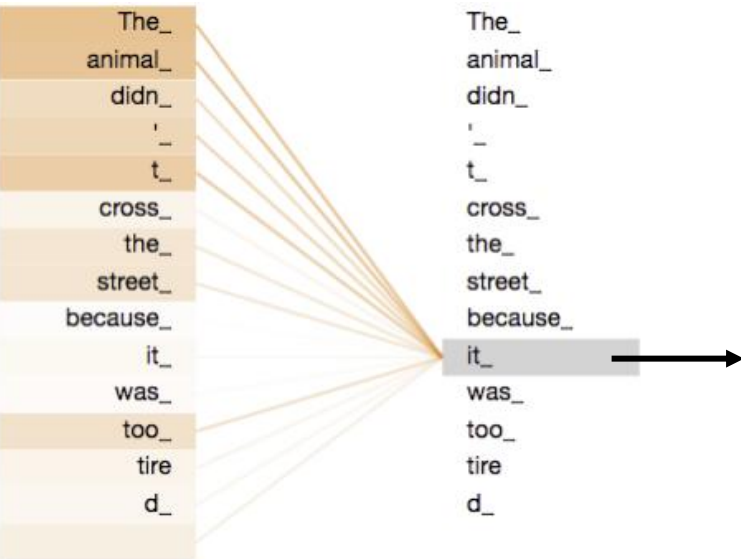
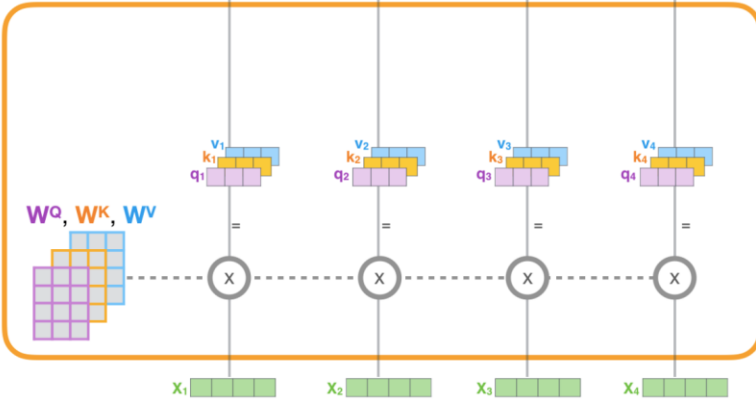
- Encoder: blocks with multi-head self-attention and feed-forward networks (MLPs).
- Decoder: blocks with masked self-attention, encoder–decoder attention, and feed-forward networks.
- Designed for sequence-to-sequence tasks (e.g., machine translation). In Vision–Language Models (VLMs), typically **only the encoder part** is used to encode text.



Self-Attention

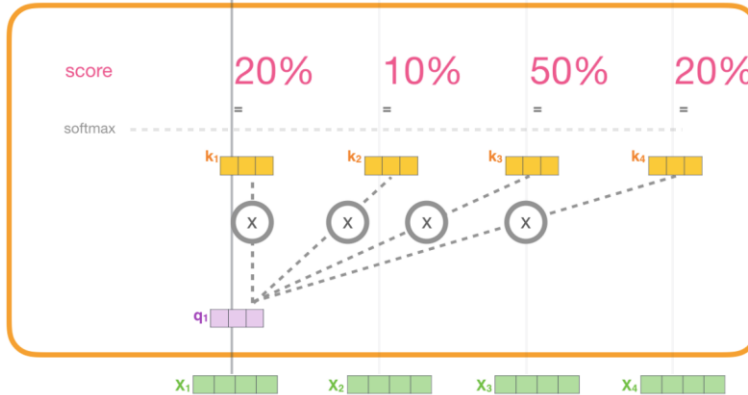
1) For each input token, create a **query vector**, a **key vector**, and a **value vector** by multiplying by weight Matrices W^Q , W^K , W^V

Self-Attention



2) Multiply (dot product) the current **query vector**, by all the **key vectors**, to get a score of how well they match

Self-Attention



Contextualized Representation

Keys

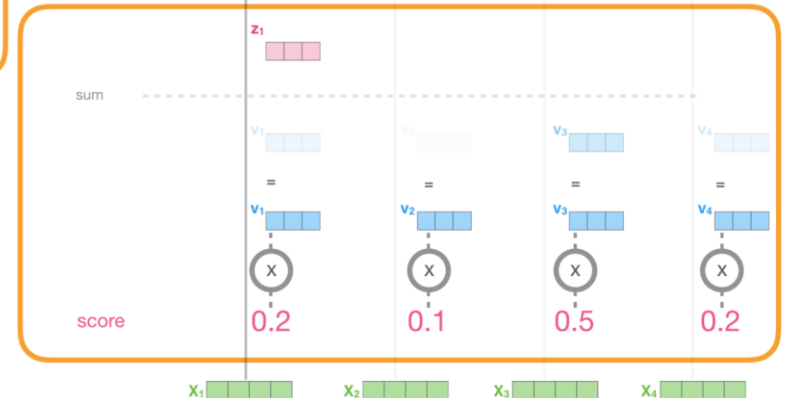
robot	must	obey	orders
robot	must	obey	orders
robot	must	obey	orders
robot	must	obey	orders

Scores (before softmax)

0.11	0.00	0.81	0.79
0.19	0.50	0.30	0.48
0.53	0.98	0.95	0.14
0.81	0.86	0.38	0.90

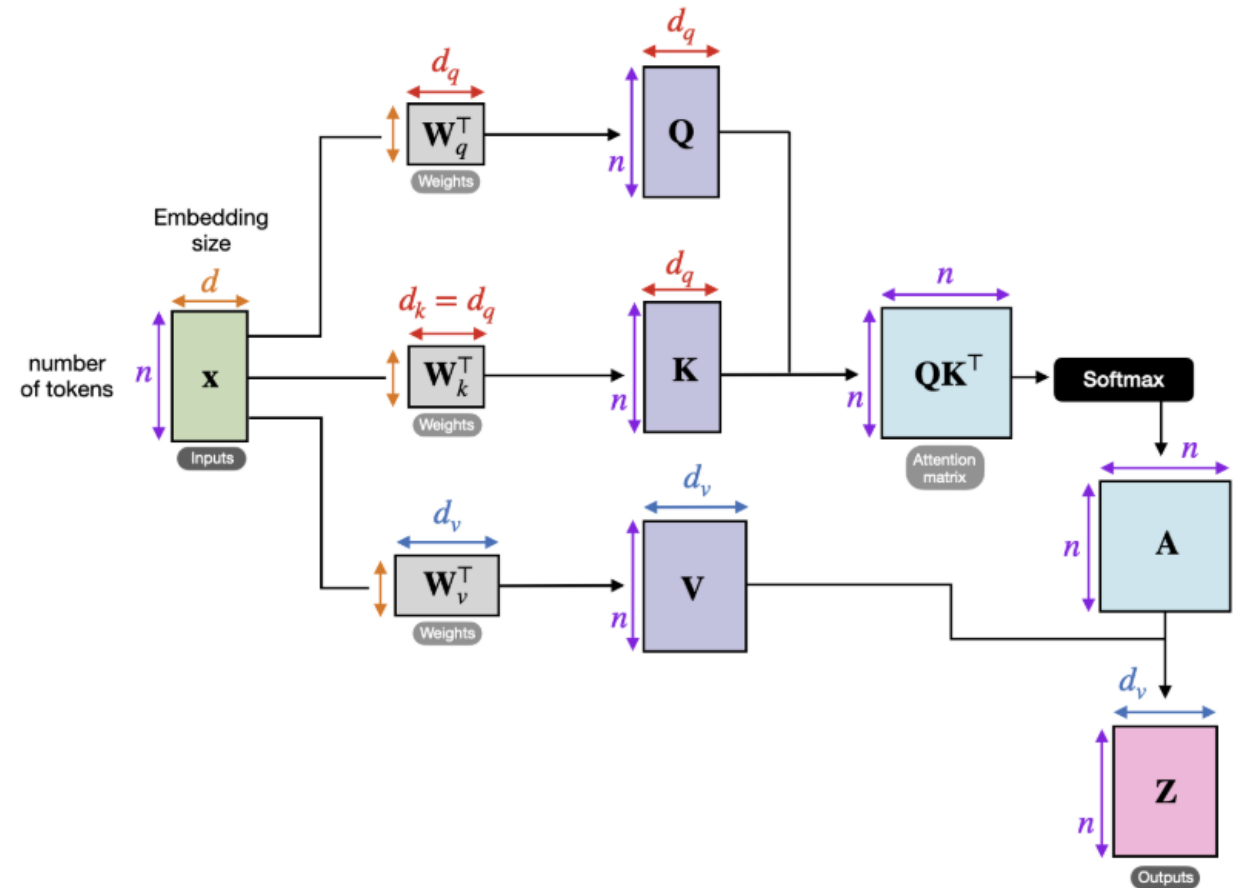
3) Multiply the **value vectors** by the **scores**, then sum up

Self-Attention



MultiHead Self-Attention

- 1. Linear projections:** Input tokens $\rightarrow$ three matrices: Query (Q), Key (K), Value (V) using learned weights W_q , W_k , W_v .
- 2. Similarity (attention scores):** Compute dot product between queries and keys:
$$\omega_{ij} = Q_i \cdot K_j^T$$
- 3. Scaling + Softmax:** Scale by d_k to stabilize gradients. Apply softmax $\rightarrow$ normalized attention weights α_{ij}
- 4. Weighted sum (context):** Use weights α_{ij} to combine values V_j . Produces context vector for each token.



MultiHead Self-Attention

5. Multi-head mechanism: Repeat above with multiple sets of W_q , W_k , W_v .

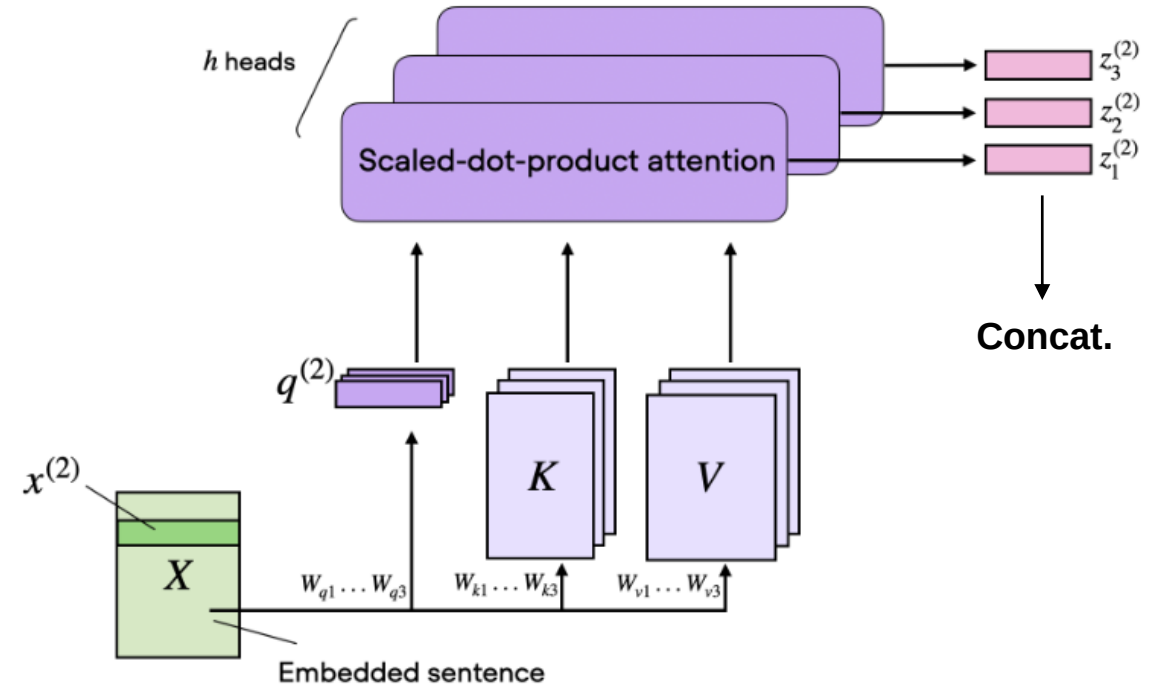
Concatenate all heads $\rightarrow$ project back to original dimension.

Example

Given:

- Input vector x has dimension 512.
- Number of attention heads = 8.
- Therefore each head will work in a smaller subspace of dimension:

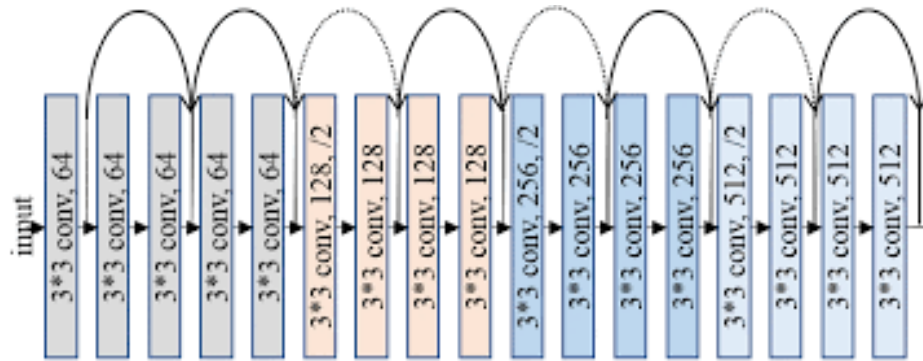
$$d_h = \frac{512}{8} = 64$$



VLMs - Vision components

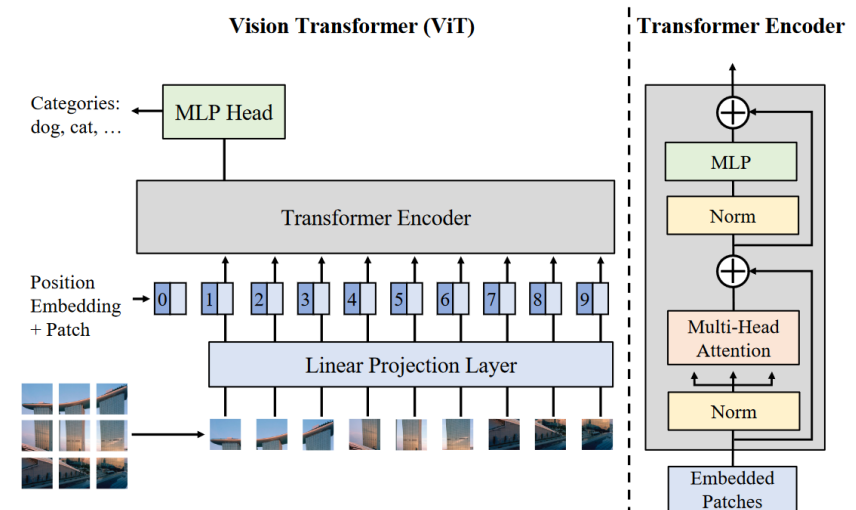
ResNet

- Based on convolutional layers for hierarchical feature extraction.
- Introduces **residual connections** (skip connections) to ease training of very deep networks.
- Enables learning by adding identity shortcuts, preventing vanishing gradients and improving accuracy.



Vision Transformer (ViT)

- Uses a stack of Transformer blocks with multi-head self-attention and feed-forward layers.
- The image is divided into fixed-size patches.
- Each patch is linearly projected and combined with positional embeddings before entering the Transformer encoder.



ResNet

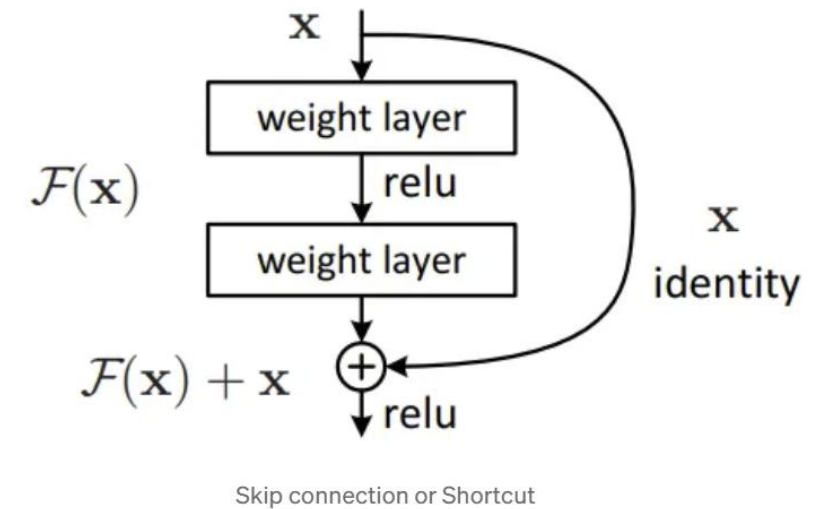
Problem: Deep CNNs suffer from vanishing/exploding gradients
→ performance worsens with depth (more layers).

Solution: Residual Blocks with skip connections let layers learn residual mapping ($F(x) = H(x) - x$) instead of direct mapping.
 $H(x)$: True target mapping.

Benefit:

- Allows very deep networks (100–1000 layers).
- Skip links act like regularization → poor layers are bypassed automatically.

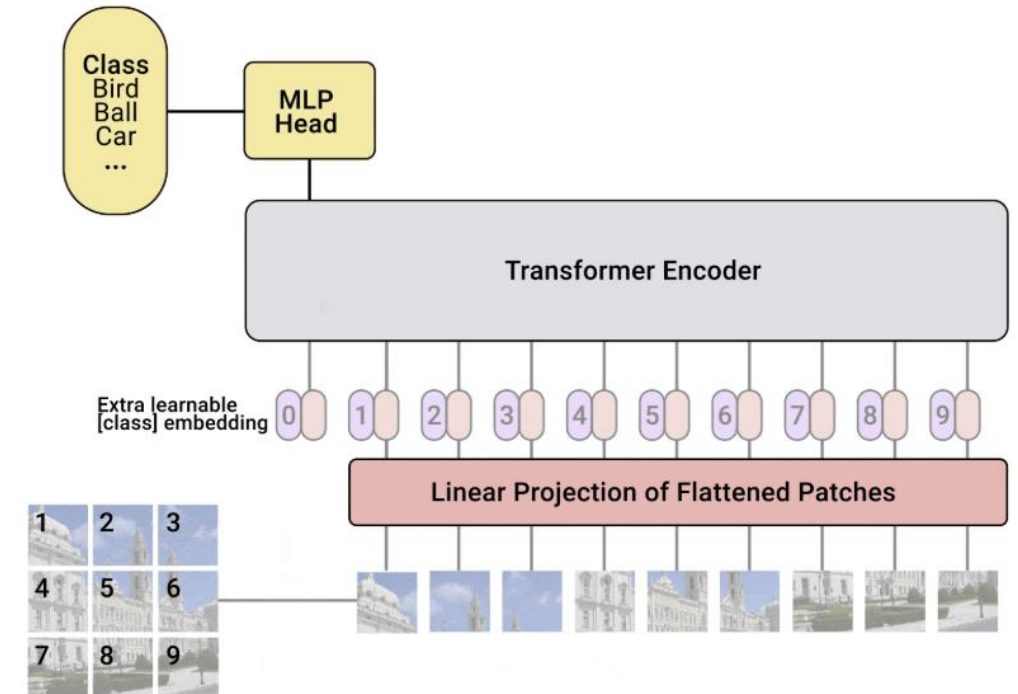
ResNetV2 vs V1: batch norm before weight layers in V2.



Vision Transformer (ViT)

Patch Projection in ViT

- 1. Image Splitting:** Input image (e.g., $224 \times 224 \times 3$) is divided into fixed-size non-overlapping patches (e.g., 16×16). For 224×224 image with 16×16 patches $\rightarrow (224/16)^2 = 196$ patches.
- 2. Flattening:** Each patch ($16 \times 16 \times 3 = 768$ pixels) is flattened into a 1D vector.
- 3. Linear Projection:** A trainable linear layer (matrix E) maps each flattened vector into a lower-dimensional embedding (e.g., $768 \rightarrow 512$).
- 4. Positional Embeddings:** Since transformers don't know about spatial order, a positional embedding is added to each patch embedding before feeding into the Transformer encoder.



CNNs vs. Vision Transformers (ViTs)

- **Inductive Bias**

CNNs: Have strong inductive biases → translation equivariance (filters slide), locality (small receptive fields first).

ViTs: Have weaker inductive bias → rely on attention to learn spatial relationships instead of being hard-coded.

Means ViTs need more data to learn well, but are more flexible.

- **Representations:** ViT intermediate features differ from CNNs (more global, less hierarchical).

- **Computation**

CNNs: Efficient for smaller images, cost grows linearly with image size.

ViTs: Self-attention has quadratic cost in number of patches → computationally heavy for high-res images.

VLMs - Examples

CLIP: Learning Transferable Visual Models From Natural Language Supervision

- Goal: learn joint image-text representations by
 - **Pulling together** paired embeddings (I, T)
 - **Pushing apart** non-matching pairs
- Symmetric InfoNCE loss:

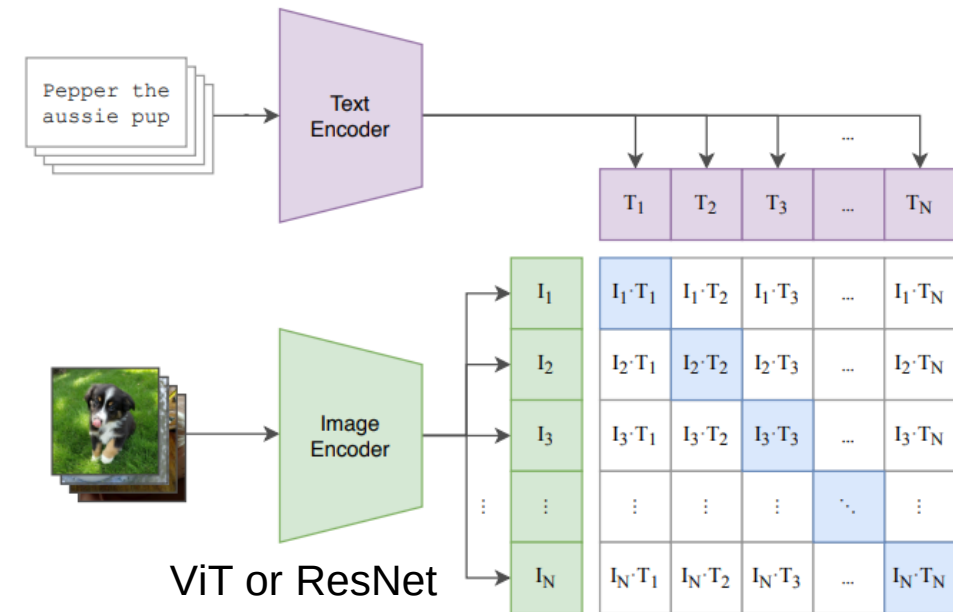
$$L_{IT} = (L_{I \rightarrow T} + L_{T \rightarrow I}) / 2$$

- For a batch of B pairs:

$$L_{I \rightarrow T} = -\frac{1}{B} \sum_{i=1}^B \log \frac{\exp(\text{sim}(I_i, T_i)/\tau)}{\sum_{j=1}^B \exp(\text{sim}(I_i, T_j)/\tau)}$$

$$L_{T \rightarrow I} = -\frac{1}{B} \sum_{i=1}^B \log \frac{\exp(\text{sim}(T_i, I_i)/\tau)}{\sum_{j=1}^B \exp(\text{sim}(T_i, I_j)/\tau)}$$

- $\text{sim}(\cdot, \cdot)$: cosine similarity
- τ : temperature parameter



Contrastive Loss

Large Batch Size

- Essential for methods like SimCLR, CLIP.
- Bigger batch → more in-batch negatives → stronger learning signal.
- Trade-off: high memory usage.

Hard Negative Mining

- Hard negatives: different labels but embeddings close to anchor.
- Easy with labels (e.g., NLI “contradiction” pairs, BM25 top mismatches).
- Unsupervised case: harder → rely on large batch/memory bank to implicitly include them.

```
# image_encoder - ResNet or Vision Transformer
# text_encoder  - CBOW or Text Transformer
# I[n, h, w, c] - minibatch of aligned images
# T[n, l]       - minibatch of aligned texts
# W_i[d_i, d_e] - learned proj of image to embed
# W_t[d_t, d_e] - learned proj of text to embed
# t            - learned temperature parameter

# extract feature representations of each modality
I_f = image_encoder(I) #[n, d_i]
T_f = text_encoder(T)  #[n, d_t]

# joint multimodal embedding [n, d_e]
I_e = np.linalg.norm(np.dot(I_f, W_i), axis=1)
T_e = np.linalg.norm(np.dot(T_f, W_t), axis=1)

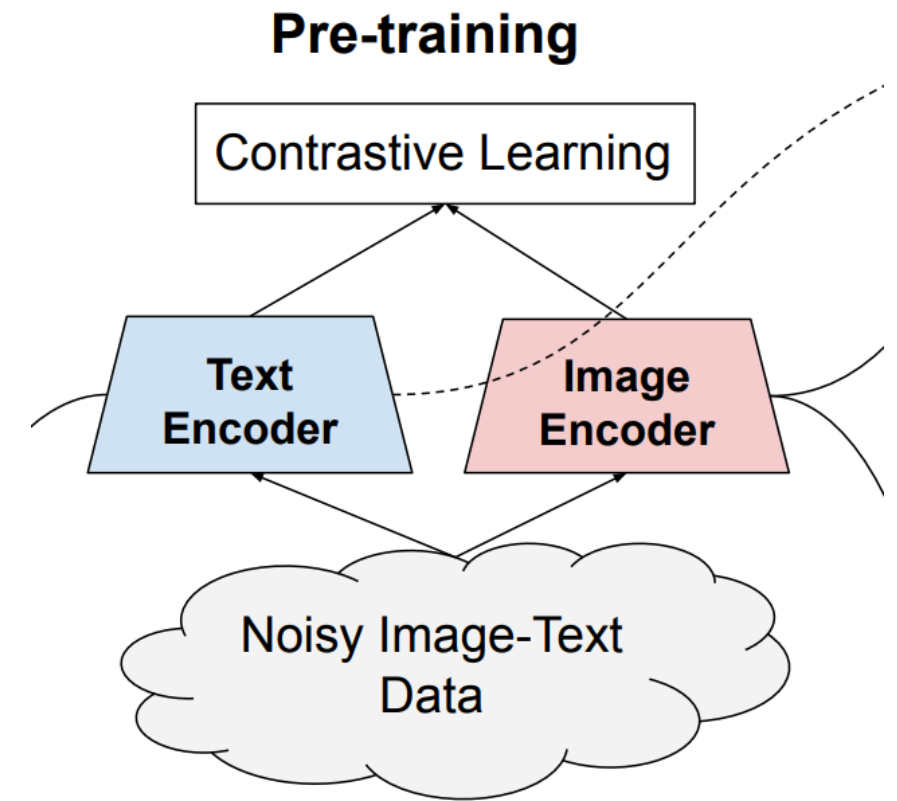
# scaled pairwise cosine similarities [n, n]
logits = np.dot(I_e, T_e.T) * np.exp(t)

# symmetric loss function
labels = np.arange(n)
loss_i = cross_entropy_loss(logits, labels, axis=0)
loss_t = cross_entropy_loss(logits, labels, axis=1)
loss   = (loss_i + loss_t)/2
```

VLMs - Examples

ALIGN: Scaling Up Visual and Vision-Language Representation Learning With Noisy Text Supervision

- Use 1B+ noisy image–alt text pairs collected from the web.
- Minimal cleaning, no heavy annotation.
- Train a dual-encoder model (image encoder + text encoder) with contrastive loss.



VLMs - Examples

CoCa: Contrastive Captioners are Image-Text Foundation Models

- Combines contrastive learning (like CLIP) with caption generation in a single framework.

1. Image-Text Contrastive Loss

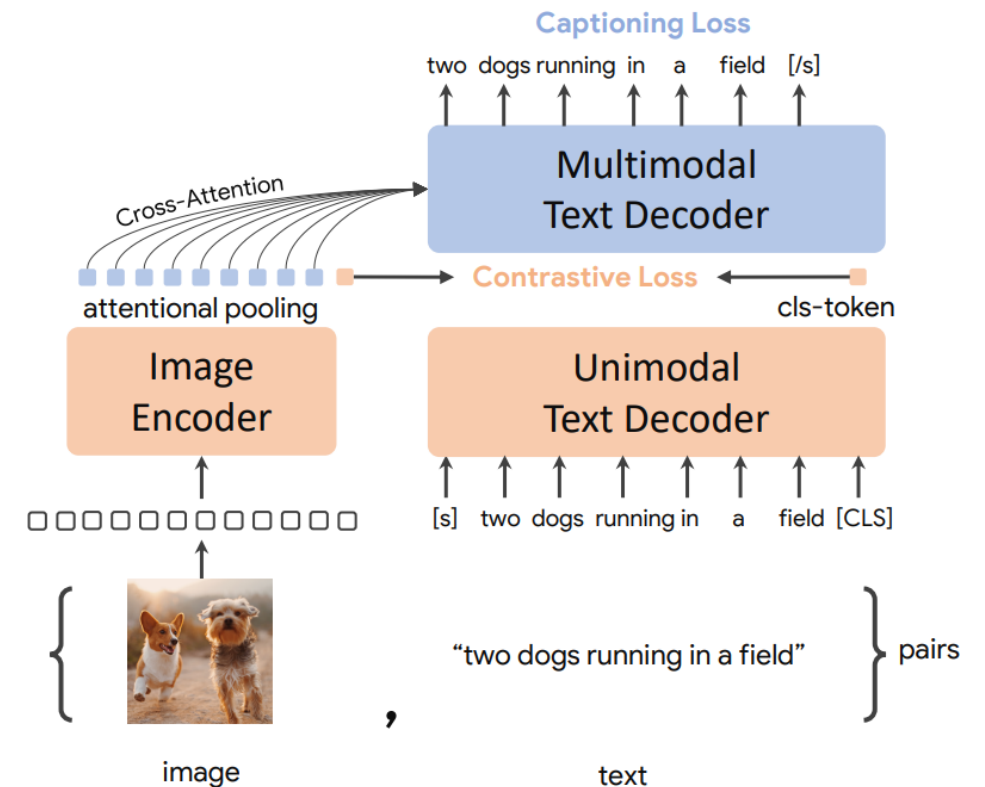
$$L_{IT} = L_{I \rightarrow T} + L_{T \rightarrow I}$$

2. Captioning Loss

$$L_{\text{cap}} = - \sum_{t=1}^T \log P(w_t \mid w_{<t}, I)$$

- Final Objective

$$L = L_{IT} + L_{\text{cap}}$$



Pre-training Datasets (scalable training)

- [COCO Caption](#): contains over 330k images from MS COCO. It has two versions: COCO Caption c5 with 5 reference captions for 330k images;
- [CC12M/CC3M](#): is introduced specifically for VLM pretraining. By relaxing the data collection pipeline used in CC3M, CC12M collects less precise but much larger size of data, i.e., 12M **image-text pairs**.
- [LAION400M](#): LAION400M consists of 400M image-text pairs filtered by CLIP, which also provides the data embeddings and kNN indices.
-



<PERSON> was the first US president to attend a tournament in sumo's hallowed Ryogoku Kokugikan arena. (AFP photo)



Hand holding a fresh mangosteen



#jellyfish #blue #ocean #pretty Sea Turtle Wallpaper, Aquarius Aesthetic, Blue Aesthetic Pastel, The Adventure Zone, Capricorn And <PERSON>, Life Aquatic, Ocean Life, Jellyfish, Marine Life



ZERO-SHOT TASKS WITH VLMs



Zero-Shot Downstream Tasks

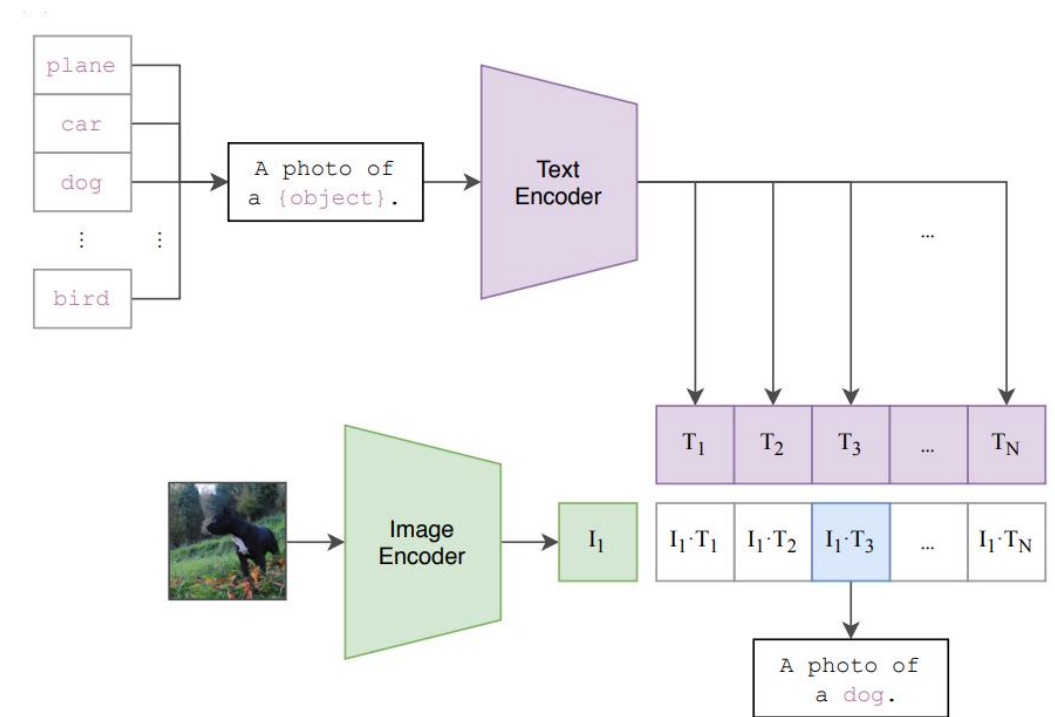
Zero-Shot Classification: No task-specific training — just reframe classification as matching image with text prompts.

How it works:

1. Define label set as **text prompts** (e.g., “a photo of a dog”, “a photo of a cat”).
2. Encode image (I) and text prompts (T) with the VLM.
3. Compute similarity (e.g., cosine similarity) between I and each T.
4. Choose the label with highest similarity.

$$\hat{y} = \arg \max_c \text{sim}(f_I(I), f_T(T_c))$$

where T_c is the prompt for class c .



Zero-Shot Downstream Tasks

Zero-Shot Crossmodal Retrieval: Retrieve the most relevant images given text (or text given image), without task-specific training.

Two settings:

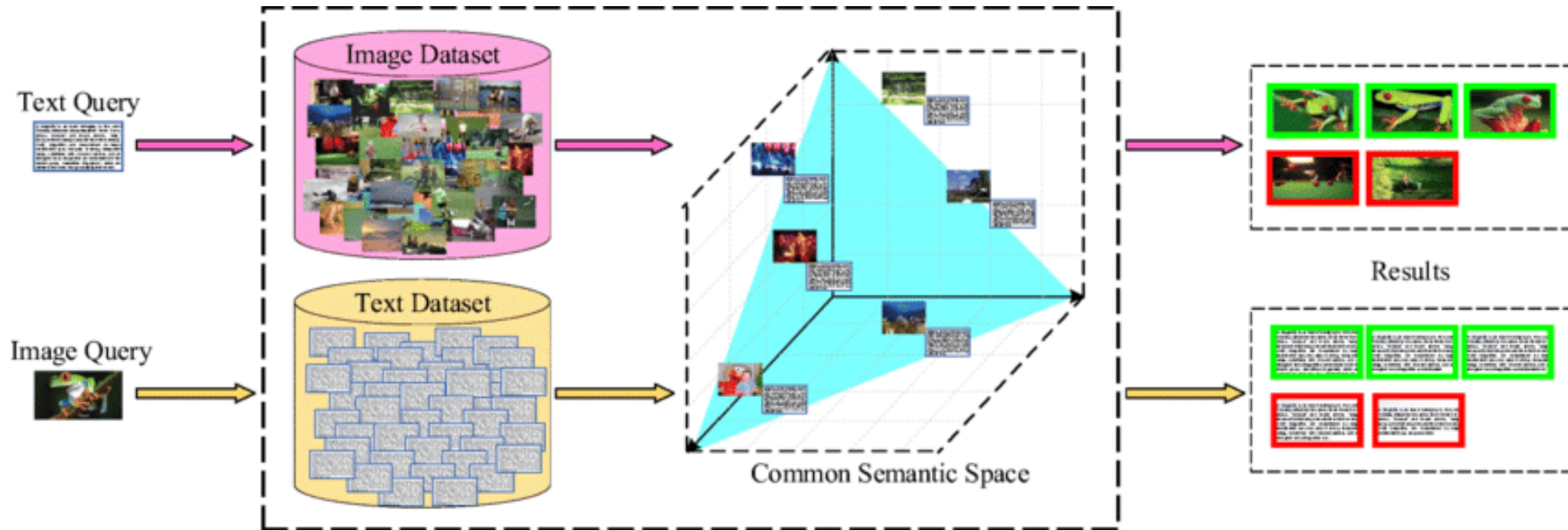
- Image → Text retrieval (captioning-like): find the best matching text for a query image.
- Text → Image retrieval (search-like): find the most relevant image for a text query.

How it works:

1. Encode query (image or text).
2. Encode all candidates (texts or images).
3. Rank by similarity in the shared embedding space.

Zero-Shot Downstream Tasks

Zero-Shot Crossmodal Retrieval: Retrieve the most relevant images given text (or text given image), without task-specific training.





EMBEDDING SPACE STRUCTURE OF VLMs



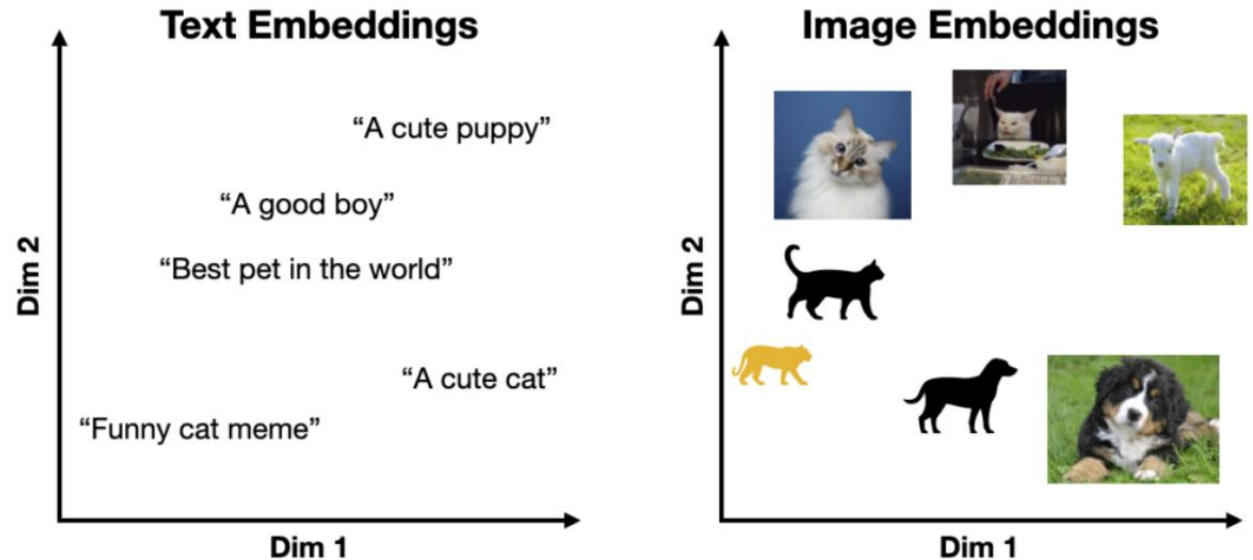
Shared Embedding Space

Embeddings: numerical representations of data learned during training.

- Example: BERT (text), ViT (images).

Structure: embedding spaces cluster similar concepts close together.

Limitation: single-modality models (only text or images) → cannot support cross-modal tasks (e.g., captioning, multimodal search).



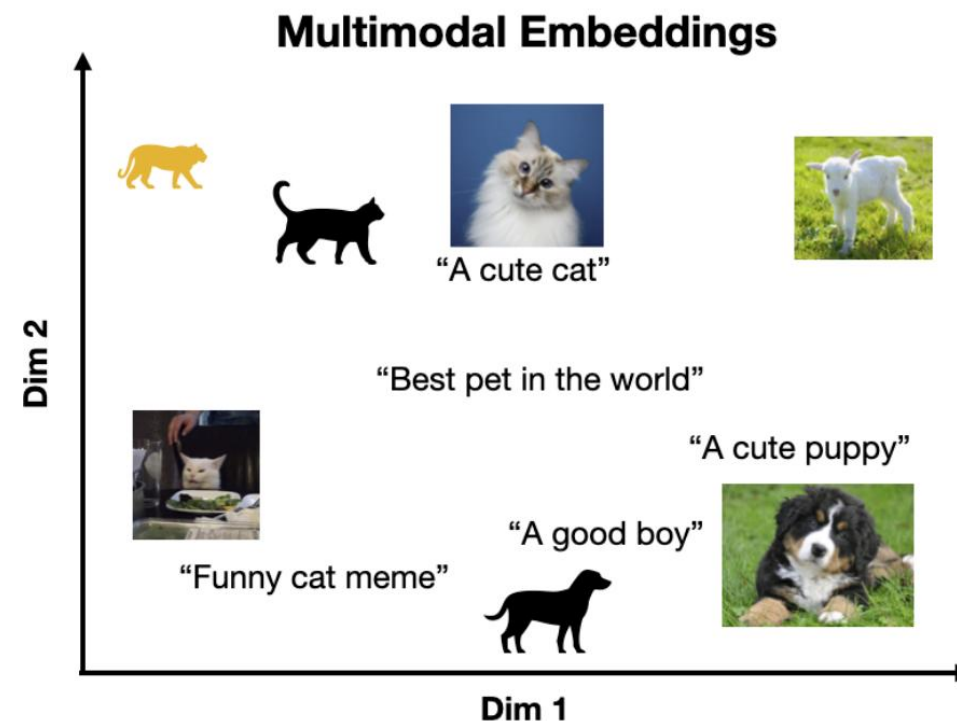
Shared Embedding Space

Key Idea: All modalities (text, images, audio, etc.) can be represented as vectors → align them into one shared space.

Multimodal Embeddings: place semantically similar concepts near each other, regardless of modality.

- Example: CLIP aligns text + image in the same space → enables zero-shot classification, retrieval, captioning.

Beyond CLIP: same principle extends to text-audio, text-video, text-EEG, etc. → unlocks rich cross-modal applications.



Modality Gap

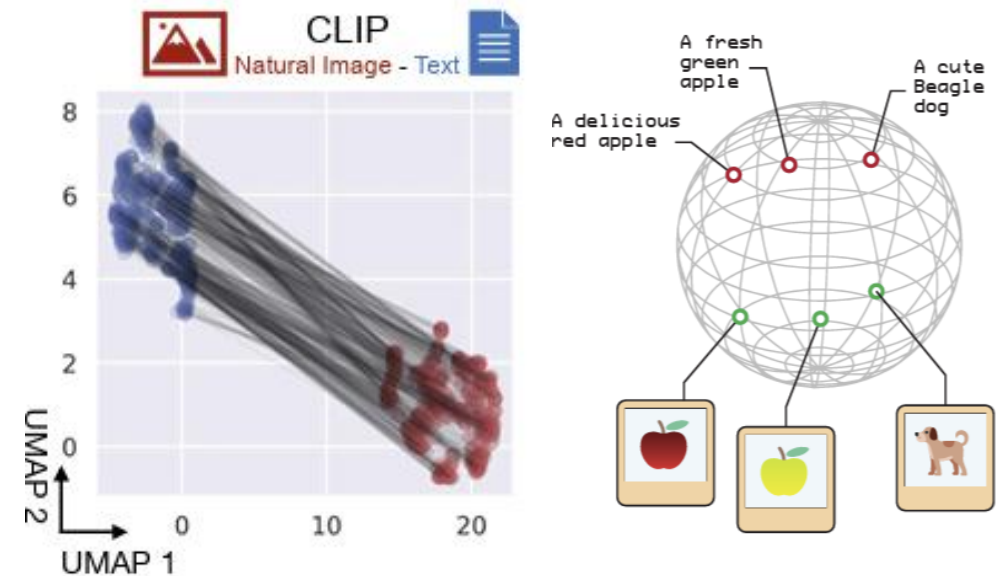
Definition: The modality gap is a geometric separation in the shared embedding space of multimodal models (e.g., CLIP), where text and image embeddings remain at a distance from each other.

Why it happens:

- Initialization: neural nets produce embeddings constrained to a narrow cone → different encoders start apart.
- Optimization (contrastive learning): the loss (controlled by temperature) keeps modalities separated.

Consequences:

- Does the size of the gap affect zero-shot task performance?



How to reduce?

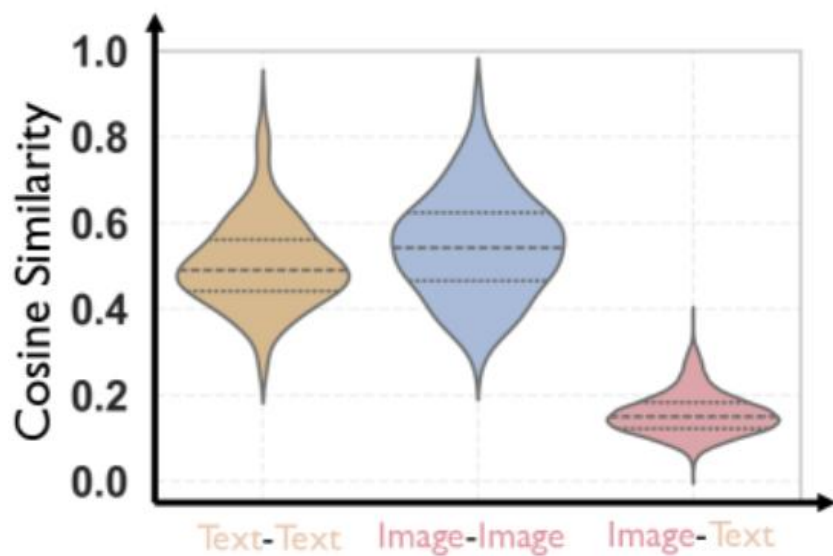
Some methods have been proposed to reduce the gap, but it remains an **open research problem.**

Modality Gap - Mixed Modality Search

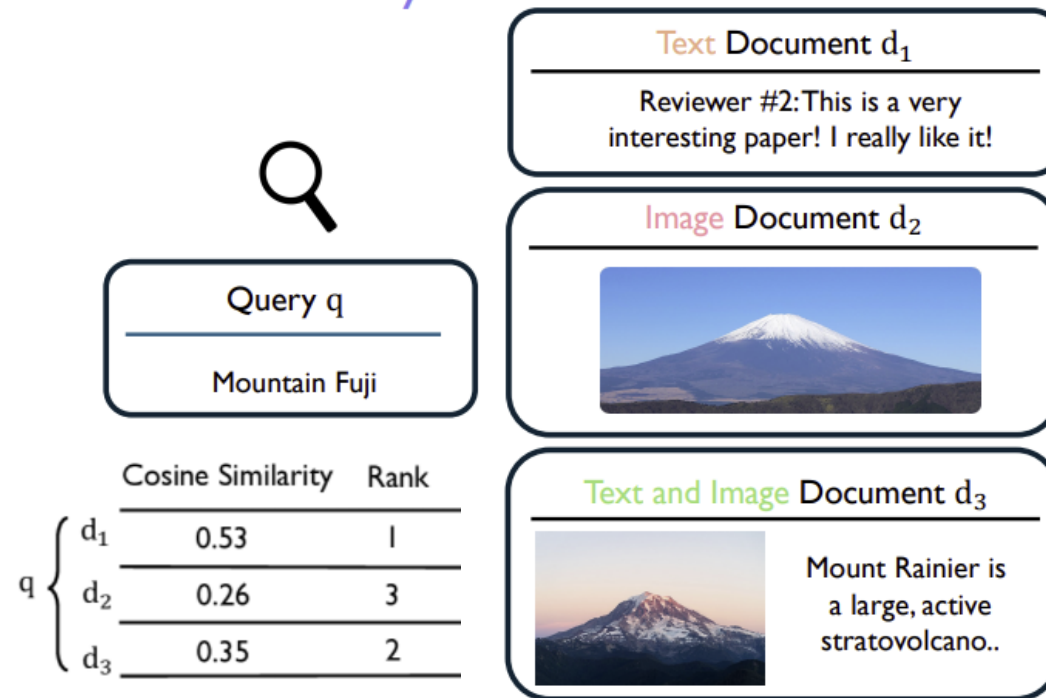
Mixed modality search: retrieving across texts, images, multimodal docs.

CLIP struggles → modality gap causes:

- Intra-modal ranking bias
- Inter-modal fusion failure



Mixed Modality Search





LINKSFOUNDATION.COM

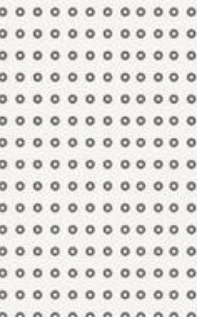


CONTACTS

Federico D'Asaro

Team Leader

1. federico.dasaro@linksfoundation.com
2. federico.dasaro@polito.it



FONDAZIONE LINKS
Via Pier Carlo Boggio 61 | 10138 Torino
P. +39 011 22 76 150
LINKSFOUNDATION.COM

